

# 中山大学计算机学院本科生实验报告

## (2024 学年第 1 学期)

课程名称: Data structures and algorithms

任课教师: 张子臻

|      |             |         |                           |
|------|-------------|---------|---------------------------|
| 年级   | 23 级        | 专业 (方向) | 计算机科学与技术                  |
| 学号   | 23320093    | 姓名      | 林宏宇                       |
| 电话   | 13421145433 | Email   | linhy69@mail2.sysu.deu.cn |
| 开始日期 | 2024.11.17  | 完成日期    | 2024.11.22                |

### 第一题

#### 1. 实验题目

交错路径

#### 2. 实验目的

给你一棵以 root 为根的二叉树，二叉树中的交错路径定义如下：

- 选择二叉树中 任意 节点和一个方向（左或者右）。
- 如果前进方向为右，那么移动到当前节点的右子节点，否则移动到它的左子节点。
- 改变前进方向：左变右或者右变左。
- 重复第二步和第三步，直到你在树中无法继续移动。

交错路径的长度定义为：访问过的节点数目 - 1（单个节点的路径长度为 0）。

请你返回给定树中最长 交错路径 的长度。

树节点数据结构定义如下：

```
struct TreeNode
{
    int val;

    TreeNode *left;

    TreeNode *right;
```

```
};
```

### 3. 算法设计

1. 开始对根节点任意选择左右方向，分别判断先朝左和朝右的最大交错路径，然后选择较大值
2. 设计计算最长交错路径的函数 dfs, 需要记录当前遍历结点，上一次遍历方向，当前最大长度

### 4. 程序运行与测试

```
1.int longestZigZag(TreeNode* root)
{
    int left=dfs(root->left,-1,0);
    int right=dfs(root->right,1,0);
    return max(left,right);
}
2.int dfs(TreeNode* root,int dir,int len)
{
    if(root==NULL)
        return len;
    if(dir==1)//上一次是朝左
        return max(dfs(root->right,1,len+1),dfs(root->left,-1,0));
    else
        return max(dfs(root->left,-1,len+1),dfs(root->right,1,0));
}
```

### 5. 实验总结与心得

1. dfs 函数看着形式简单，但是要理解其逻辑难度较大。
2. 递归函数的终止条件为当前结点为空，返回到其的最长路径长。同时注意 dir 和 len 不能取引用，这里包含了一个回溯的逻辑。
3. 每一次的递归为：比较 继续走交错路径的长度与不走交错路径的长度，取他们的最大值返回

## 第二题

### 1. 实验题目

判断二叉查找树是否平衡

### 2. 实验目的

问题

给定一颗二叉查找树，其中结点上存储整数关键字，请你判断它是否一棵平衡的二叉查找树，即每个内部结点的两颗子树高度差小于等于 1。

输入

第一行是测试样例数，接下来是若干测试样例。对于每个测试样例，第一行是二叉树结点数  $n$ ，第二行是  $n$  个关键字序列，表示二叉树的先序遍历结果。假定关键字均不相同。

输出

如果二叉查找树是平衡的，则输出 “YES”，否则输出 "NO",每个输出占一行。

### 3. 算法设计

1. 输入与输出部分
2. 判断 YES or NO，利用先序遍历的特点，当数组长度大于等于 3 时先递减再递增
3. 分情况分析
  - a. 只有一个数时，直接打印 YES；
  - b. 只有两个数时，第一个数大于第二个数，就打印 YES，否则 NO
  - c. 当有三个数时，需要判断数组是否先递减再递增即 YES，否则 NO

### 4. 程序运行与测试

1. a. 只有一个数时

```
if(num<=1)
{
    cout<<"YES"<<endl;
    continue;
}
```

2. b. 只有两个数时

```
else if(num==2)
```

```

{
    if(v[0]>v[1])
        cout<<"YES"<<endl;
    else
        cout<<"NO"<<endl;
    continue;
}

```

### 3. c.大于等于三个数时

```

if(v[0]>v[1] && v[num-1]>v[num-2])
{
    int flag=0;
    for(int i=1;i<num;i++)
    {
        if(flag==0)
            if(v[i]>v[i-1])
                flag=1;
        else if(flag==1)
            if(v[i]<v[i-1])
                flag=2;
        else
            continue;
    }
    if(flag==1)
        cout<<"YES"<<endl;
    else
        cout<<"NO"<<endl;
}
else
    cout<<"NO"<<endl;
continue;

```

## 5. 实验总结与心得

1. 题目难度不大，较简单
2. 要重点知道平衡二叉树的先序遍历结果为先递增后递减
- 3.对数组判断是否为先递增后递减，简单的判断语句即可实现

## 第三题

### 1. 实验题目

完全二叉树-最近公共祖先

### 2. 实验目的

如下图，由正整数 1, 2, 3, ... 组成一棵无限大的满二叉树。从某一个结点到根结点（编号是 1 的结点）都有一条唯一的路径，比如 10 到根节点的路径是(10,5,2,1)，由 4 到根节点的路径是(4,2,1)，从根结点 1 到根结点的路径上只包含一个结点 1，因此路径是(1)。

对于两个结点 X 和 Y, 假设它们到根结点的路径分别是(X1,X2,...,1)和(Y1,Y2,...,1)(这里显然有 X=X1,Y=Y1), 那么必然存在两个正整数 i 和 j, 使得从 Xi 和 Yj 开始,  $X_i=Y_j, X_{i+1}=Y_{j+1}, \dots$ , 现在的问题就是, 给定 X 和 Y, 要求 Xi(也就是 Yj)。

### 3. 算法设计

1. 输入与输出

2. 对于任意一个数字 n, 可以  $n/2$  找到它的父亲结点 3. 对于  $n_1 \neq n_2$ , 可以不断对较大的那个数进行  $/2$  找到父亲结点 知道  $n_1 = n_2$  就找到了公共祖先

### 4. 程序运行与测试

```
1.  int n;
    cin>>n;
    while(n-->0)
    {
        int num1=0;
        int num2=0;
        cin>>num1>>num2;
        while(num1!=num2)
        {
            if(num1>num2)
                num1/=2;
            else
                num2/=2;
        }
        cout<<num1<<endl;
```

}

## 5. 实验总结与心得

1. 实验题比较简单
2. 主要的难点在于掌握父子结点的位置关系，一个结点  $k$ ，它的孩子结点为  $2*k$  和  $2*k+1$ ，它的父亲结点为  $k/2$
3. 对于  $n_1, n_2$ ，可以不断对较大的那个数进行  $/2$  向上找公共祖先结点